AI-Assisted Coding: From Idea to Working Prototype

VS Code, GitHub Copilot and three POCs with Streamlit, FastAPI, SQLite & XGBoost

Prof. Dr. Christoph Weisser

AI-Driven Automation • Bielefeld School of Business, HSBI

Winter Semester 2026/27

Slides, notebooks and data: github.com/ChrisW09/Python-for-AI-Driven-Automation

About your lecturer



Prof. Dr. rer. pol. Christoph Weisser

MLitt (St Andrews), MSc (Oxford)

- Professor of Mathematics, in particular Business Data Science, at HSBI
- Former Technical Lead for Artificial Intelligence and Industrial PhD Supervisor at BASF
- **Research:** Data Science, Machine Learning and Artificial Intelligence
- **Focus:** application of AI in business and industry

christoph.weisser@hsbi.de

github.com/ChrisW09/Python-for-AI-Driven-Automation

Your journey in this module

From a live demo to three working prototypes – and on to LLMs and agents

Session 1

1 Live demo
idea → app in ten minutes

2 Setup
GitHub, Copilot, Git, Python, VS Code

Session 2 and following

3 Method
Git, prompts, review

4 How apps work
frontend, backend, database

5 POC 1
Streamlit data app

6 POC 2
+ FastAPI + SQLite

7 POC 3
+ XGBoost model

8 Wrap-up: from POC to production → **Next deck:** LLMs, RAG & Agentic AI

Course repository: `github.com/ChrisW09/Python-for-AI-Driven-Automation` – notebooks, exercises, data, slides

LLM access for your projects: we provide a **course API key** for smaller and larger LLMs – you will receive it **by email**. Treat it like a password: never pass it on or put it on GitHub.

Learning objectives

After this part of the module you can ...

- set up a professional toolchain: **VS Code, GitHub Copilot, Git, Python**
- use the Copilot **agent** to build small apps from a precise description
- build a **Streamlit** data app on your own
- explain and implement a **three-tier architecture** (Streamlit + FastAPI + SQLite)
- build an ML application end to end: data → model → API → UI
- **read, test and improve** AI-generated code instead of trusting it blindly
- run a clean **Git/GitHub workflow** for your prototypes

The goal is not to become a software engineer – it is to turn business ideas into working prototypes you understand.

What this course is *not*

Deliberately out of scope – so we can focus on what matters for prototypes

✕ Not a software-engineering course

No unit-test suites, no CI/CD pipelines, no production deployment.

✕ Not a frontend course

React, Vue or Tailwind appear only as an outlook.

✕ Not an ML theory course

XGBoost is applied and evaluated, not derived mathematically.

✕ Not a DevOps course

Docker, Kubernetes and cloud deployment come later – if at all.

We build *small, running* prototypes that answer a question – not products.

Contents

1. From idea to app in ten minutes
2. Setting up your toolchain
3. Working with a coding agent
4. How apps are built
5. POC 1 – A data app with Streamlit
6. POC 2 – Frontend, backend and database
7. POC 3 – Adding a machine-learning model
8. Wrap-up

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

Part 1 • Session 1: From idea to app in ten minutes

A live demo first – the theory comes afterwards.

- ✔ See what a coding agent can build from one precise description
- ✔ Understand where you, the human, stay in charge
- ✔ Know the difference between "vibe coding" and AI-assisted coding

Let's build something – live

A small automation for a finance team: the expense-claim checker

Today: a manual routine

Every month, a controller checks a few hundred expense claims line by line against the travel policy.

Goal of the demo

An app that does the *first check*: upload claims, see flagged items with a reason, download the list.

Rules the app should check

Rule	Example
Category limit	meals > 60 €, taxi > 80 €, hotel > 180 € per night
Weekend	claim dated on a Saturday or Sunday
Duplicate	same employee, date and amount twice

Limits are illustrative, not a real policy.

Pick a boring, rule-based check – the perfect candidate for a first automation prototype.

The one prompt

Everything the agent needs – goal, data, rules, output, deliverables

Demo prompt for Copilot – Agent (typed live)

Build a small Streamlit app `app.py` that automates the first check of employee expense claims. First create `sample_data.py`, which generates about 300 realistic demo claims (employee, department, date, category, amount in EUR, description) with some problems. The app loads the demo data or an uploaded CSV and flags claims that (1) exceed the category limit (meals 60, taxi 80, hotel 180 EUR per night), (2) fall on a weekend, or (3) look like duplicates (same employee, date and amount). Show KPIs, a table of flagged claims with the reason, a bar chart of flagged amounts per department and a CSV download of the flagged claims. Add `requirements.txt` and a short README, create a virtual environment `.venv`, install the requirements there and start the app.

Goal & data

what to build, demo data

Rules & output

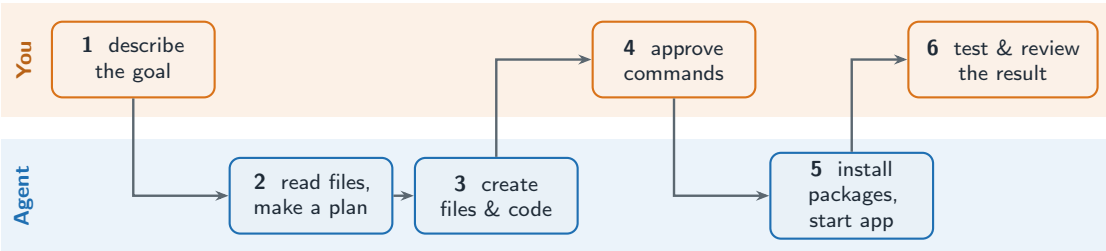
testable rules, concrete UI

Deliverables

files, packages, run it

What just happened?

You decided, the agent executed – and you checked



The agent's tools

read and edit files • run terminal commands • read error messages and try again

Your job

a precise goal • conscious approvals • checking that the rules are *really* implemented

Why this matters for AI-driven automation

Test ideas cheaply

A clickable prototype in hours: learn early whether an automation idea is worth it.

The bottleneck shifts

From *typing code* to *specifying* and *verifying* – core business skills.

A shared language

A running prototype says more than twenty pages of requirements.

But: a prototype is not a product

Security, data protection, tests and operations still need professionals – see the wrap-up.

Business people who can prototype become better partners for IT – not replacements for it.

"Vibe coding" vs. AI-assisted coding

Same tools, very different attitude

> **Vibe coding** (Karpathy, 2025)
> "[. . .] fully give in to the vibes, embrace exponentials, and forget that the code even exists."

> **AI-assisted coding** (Willison, 2025)
> "[If you] reviewed it, tested it thoroughly and made sure you could explain how it works [. . .] that's not vibe coding, it's software development."

	Vibe coding	AI-assisted coding
You read and understand the changes	✗	✓
You run and test it, including an error case	✗	✓
Every working step is committed to Git	✗	✓

Our rule: you own every line you run or share – no matter who (or what) typed it.

Source: Karpathy (2025); Willison (2025)

What the evidence says

AI assistance is powerful – and uneven

↑ Well-defined tasks

In controlled experiments, people with an AI assistant finish clearly specified tasks much faster.

Peng et al. (2023); Noy & Zhang (2023)

↓ Large, familiar code

Experienced developers working in their own large projects were *slower* with AI tools – while believing they were faster.

Becker et al. (2025)

The jagged frontier

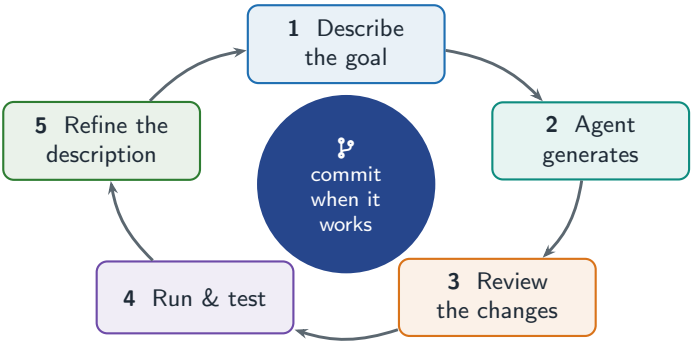
Inside the AI's capability frontier, quality rises; just outside it, people who rely on AI make more mistakes.

Dell'Acqua et al. (2026)

Using AI well is a skill: know when to delegate, when to check – and when to think for yourself.

The AI-assisted coding loop

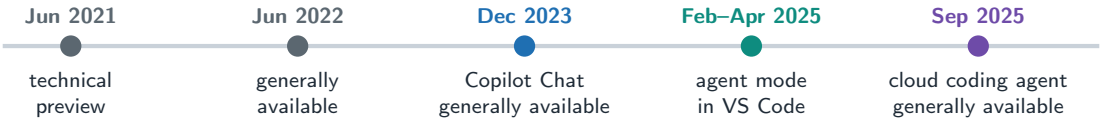
Seconds to minutes per iteration – with a commit whenever something works



Small steps win: one feature per loop, test it, commit it – then the next one.

From autocomplete to agents

Five years of GitHub Copilot



Gen. 1: autocomplete

AI completes the line or function you are typing.

Gen. 2: chat

AI answers questions, explains and rewrites code blocks.

Gen. 3: agents

AI plans, edits many files, runs commands and iterates – *with your approval*.

What is a proof of concept (POC)?

And what it deliberately leaves out

Proof of concept

- a **running prototype** that proves one concrete idea
- small and focused – built in hours to days
- optimised for **learning speed**, not robustness
- just enough to answer one question – no more

Deliberately left out

tests, authentication, multiple users, caching, background jobs, deployment, visual polish

Typical POC questions

Can we detect this automatically? Is the data good enough? Would users accept this workflow?

Rule of thumb: better to prove three things quickly than to perfect one thing slowly.

What we will build

Three POCs – each one adds architecture, nothing is taken away

POC 1 • Data app

Streamlit only

upload a CSV, descriptive statistics, charts

one process, one file

POC 2 • Three tiers

Streamlit + **FastAPI** + **SQLite**

same use case, professional structure

two processes, a REST API, a database

POC 3 • ML app

+ an **XGBoost** model

predict customer churn, log every prediction

data → model → API → UI



Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

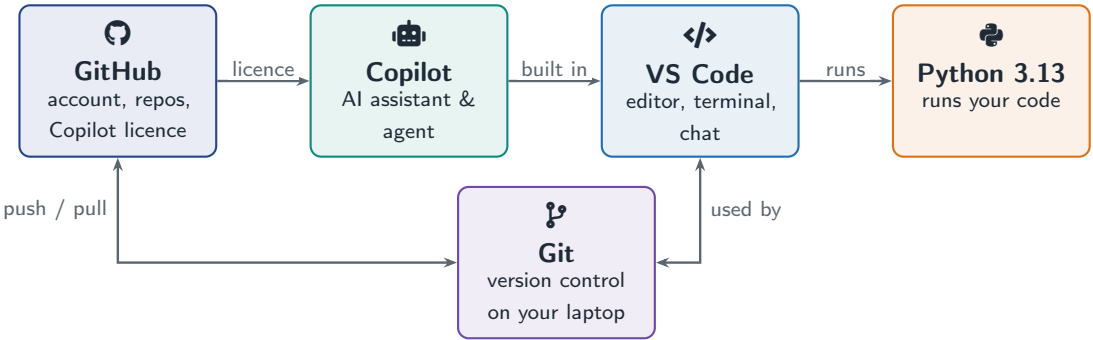
Part 2 • Session 1: Setting up your toolchain

Seven steps, in this order – we do them together in class.

- ✔ Create a GitHub account and apply for GitHub Education
- ✔ Install Git, Python 3.13 and VS Code
- ✔ Sign in to Copilot and run a first test
- ✔ Know the chat controls that keep you in charge

Your toolchain at a glance

Five tools, one job each



GitHub stores your code and your licence; VS Code is where you work; Git keeps every version.

Before class: start today

Verification takes days, not minutes

1 Create a GitHub account

`github.com/signup` – ideally with your HSBI email address; switch on two-factor authentication.

2 Apply for GitHub Education

Verification usually takes a few days – longer at the start of the semester. Apply as soon as your account exists.

No approval yet? No problem.

Every account can use **Copilot Free** at once. After approval, activate Copilot Student yourself (Education benefits → *Learn more*) – never pay.

Do both today – verification takes days, and Copilot Free bridges the gap.

Seven steps – in this order

Later steps depend on earlier ones

	Step	Time
1	GitHub account (with 2FA)	5 min
2	Apply for GitHub Education	10 min
3	Install Git	5–10 min
4	Install Python 3.13	5–10 min
5	Install VS Code + Python ext.	5 min
6	Sign in to Copilot in VS Code	2 min
7	First test with Copilot	10 min

Definition of done

You can run a Python script in VS Code *and* Copilot answers a question in the chat.

Work in pairs

One person installs, the other reads the next step aloud – then swap. Problems are solved twice as fast.

STEP 1 Create your GitHub account

1. Go to `github.com/signup`
2. Register with your **HSBI email address** (you can add a private address later)
3. Choose a **professional username** – it will appear in your CV and portfolio
4. Confirm your email address
5. Enable **two-factor authentication** (Settings → Password and authentication)

Good usernames

`cweisser`, `christoph-weisser`
not: `xX_coder99_Xx`

Why 2FA now?

GitHub Education requires it – and it protects your code and your licence.

Your GitHub profile is part of your professional identity – treat it like LinkedIn.

STEP 2 Apply for GitHub Education

Verified students get the **GitHub Copilot Student** plan

How to apply

1. Open `github.com/settings/education/benefits`
2. *Start an application*, select HSBI
3. Upload **proof of enrolment** (dated)
4. Allow **location + camera**; **no VPN**

What you get (as of Sep 2026)

unlimited code completions • monthly allowance of AI credits for chat and agent • automatic model selection

Common rejections

blurry or undated document • name differs from your GitHub billing info • VPN switched on

Until approval arrives, simply use Copilot Free.

STEP 3 Install Git

The version-control program on your laptop

Windows

Installer from git-scm.com (default options), or in a terminal:

```
winget install --id Git.Git -e --source winget
```

macOS

Apple's Command Line Tools include Git:

```
xcode-select --install
```

Then, in a new terminal (PowerShell / Terminal): check and introduce yourself

```
git --version                                  # e.g. git version 2.5x
git config --global user.name  "Firstname Lastname"
git config --global user.email "you@hsbi.de"    # same address as on GitHub
```

STEP 4 Install Python 3.13

Not the newest version – the one all our packages support

Windows

From python.org/downloads: the *Python install manager*, then:

```
py install 3.13
```

Classic installer: tick "Add python.exe to PATH".

macOS

Download the macOS installer for **Python 3.13.x** from python.org/downloads and run it.

On macOS the command is usually `python3`.

Check in a new terminal

```
python --version    # macOS: python3 --version
Python 3.13.x
```

Why not 3.14 or 3.15?

Brand-new versions such as 3.15 often lack ready-made packages (e.g. PyTorch). Python 3.13 works with everything we use.

STEP 5 Install VS Code and the Python extension

1. Download from `code.visualstudio.com` and install with the default options
2. Open the **Extensions** view (`Ctrl+Shift+X` / `Cmd+Shift+X`)
3. Install **Python** by Microsoft (brings Pylance and the debugger)
4. Optional: **Jupyter** by Microsoft

Good news

GitHub Copilot is **built into VS Code**: there is no separate Copilot extension to install any more.

Windows

Keep the option "Add to PATH" ticked – then you can open any folder with `code .`

VS Code is your cockpit: files on the left, code in the middle, terminal below, chat on the right.

STEP 6 Sign in to GitHub Copilot

1. Hover over the **Copilot icon** in the Status Bar (bottom right)
2. Choose **Use AI Features**
3. Sign in with your **GitHub account** in the browser and allow VS Code
4. Back in VS Code: open the chat with `Ctrl+Alt+I` / `Ctrl+Cmd+I`

Check your plan

On github.com: profile picture → *Copilot settings* shows whether you are on *Student* or *Free*.

Privacy

Don't paste personal or confidential company data into the chat.

No subscription yet? Signing in enrols you in Copilot Free automatically.

STEP 7 First test: environment, script, Copilot

Create a folder `hello-poc`, open it via *File → Open Folder*, then *Terminal → New Terminal*:

```
python3.13 -m venv .venv          # macOS
source .venv/bin/activate         # macOS
py -V:3.13 -m venv .venv          # Windows
.venv\Scripts\Activate.ps1        # Windows
```

Create `hello.py` and run it

```
print("Hello -- my setup works!")
```

```
python hello.py
```

Now try Copilot

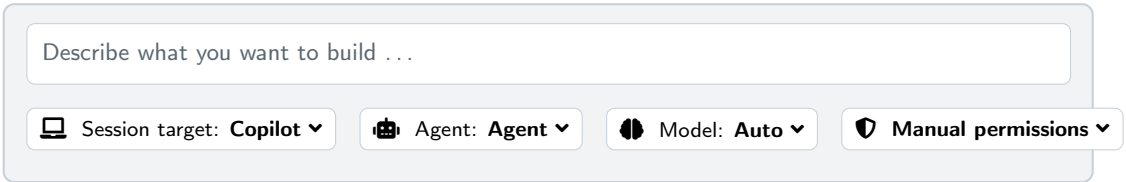
Inline: type `# function that returns the VAT for a net price` and press `Enter` – accept the grey suggestion with `Tab`.

Chat (Ask): "Explain `hello.py` and what a virtual environment is."

VS Code asks which Python to use? Pick the one inside `.venv`.

The chat view at a glance

Four controls below the input box decide how the agent works



Where it runs.
Start with *Copilot*.

Which mode.
Agent or *Plan* (`/plan`).

Which LLM.
Auto for students.

What needs your OK.
Always *Manual* in class.

Beginner settings: Copilot • Agent (or Plan) • Auto • Manual permissions.

Ask, Plan or Agent?

Choose the mode that matches the job

Mode	What it does (VS Code docs)	Use it for ...
ASK	"asks questions and provides guidance without making changes to the code"	understanding code and errors (in the *Local* target)
PLAN	"researches a task and creates a structured implementation plan before code changes"	bigger steps: review the plan first
AGENT	"autonomously plans and performs complex coding tasks, edits files, runs commands, and iterates on results"	building and changing the POCs

Ask for questions, *Plan* before big changes, *Agent* for tasks.
After the plan: *Implement Plan*.

Source: VS Code documentation (Sep 2026)

Stay in control: approvals, checkpoints, commits

Approvals

With *Manual permissions* the agent asks before risky actions. Read each command before you click *Allow*.

Checkpoints

Restore Checkpoint resets files and chat to an earlier request – but does **not** undo terminal commands.

Git commits

Your real safety net: commit after every working step, and you can always go back.

Not in this course: "Allow all" and "Autopilot"

They skip all confirmations – including file edits, terminal commands and external tools.

The agent is fast; you are responsible. Approve consciously, commit often.

Source: VS Code documentation: Approvals; Review code edits (Sep 2026)

Troubleshooting: the top six

Symptom	Fix
`python`/`git` not found	reopen terminal + VS Code; Windows: PATH option; macOS: `python3`
VS Code uses the wrong Python	`Ctrl+Shift+P` → *Python: Select Interpreter* → choose `.venv`
PowerShell: "running scripts is disabled"	`Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`
Copilot says "no access"	sign out and in again; until approval, Copilot Free is active
Education application rejected	clear, dated document; billing name as on it; no VPN; location allowed
`pip install` fails	is `(.venv)` shown? Windows: create it with `py -V:3.13`

Read the error message aloud – or paste it into Copilot Ask. Most fixes take a minute.

Setup done?

Tick every box before Session 2

- ☑ GitHub account with 2FA
- ☑ Education application submitted
- ☑ `git --version` works, name set
- ☑ `python --version`: 3.13.x
- ☑ VS Code + Python extension
- ☑ Signed in to Copilot
- ☑ `hello.py` runs in a `.venv`
- ☑ Copilot answered in Ask

Homework for Session 2

Ask Copilot (Ask) to explain three terms, then put them in your own words: *repository*, *virtual environment*, *terminal*. Write down one question you still have.

Congratulations – you now have the same toolchain as professional developers.

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

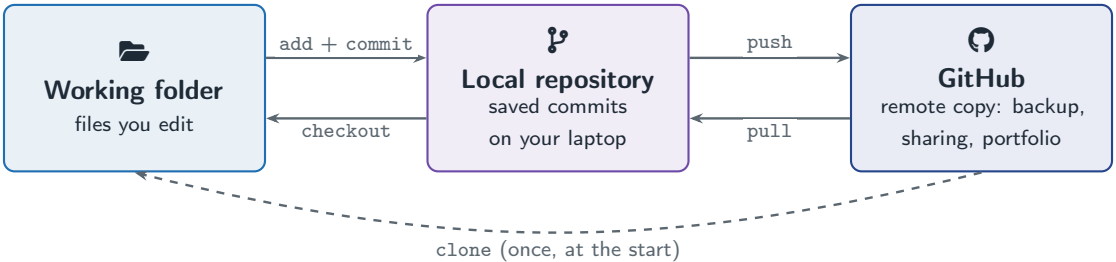
Part 3 • Session 2 and following: Working with a coding agent

The habits that make AI-assisted coding reliable.

- ✔ Use Git and GitHub as your safety net and portfolio
- ✔ Keep projects clean: environments, `.gitignore`, no secrets
- ✔ Write prompts that produce *your* code, not generic code
- ✔ Review AI-generated code in three stages

Git and GitHub in five minutes

A time machine for your project – locally and in the cloud

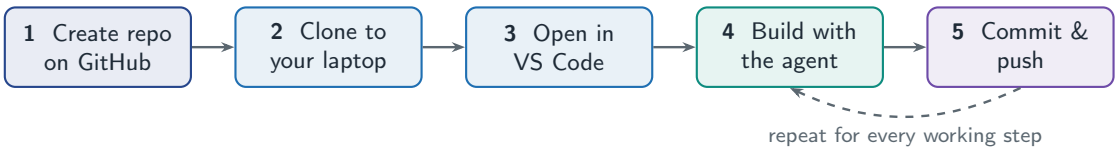


repository	project folder with its full history	**commit**	a saved snapshot with a message
push / **pull**	upload to / download from GitHub	**clone**	copy a GitHub repo to your laptop

Commit = save point. Push = backup. Both cost seconds and save hours.

Our workflow: GitHub first

Create the repository online, then work locally



Why this order?

- README, `.gitignore` (template "Python") and licence are created on GitHub with one click
- the connection to GitHub exists from the start – no `git remote add` later
- issues, branches and collaboration work from minute one

Git in practice

Terminal commands – or the same actions as buttons in VS Code

```
# 1. copy the repository to your laptop (once)
git clone https://github.com/<user>/<repo>.git
cd <repo>

# 2. ... let the agent build, review, test ...

# 3. save a working step and upload it
git status                          # what changed?
git add .
git commit -m "POC 1: CSV upload and statistics"
git push
```

Golden rule

Commit after *every* change that works.

Good commit messages

say *what* works now:
"POC 2: /api/upload stores metadata"
not: "changes", "asdf"

Commit small and often – every commit is a save point you can go back to.

One repository, three POCs

Each POC is self-contained – with its own environment and requirements

```
ai-automation-pocs/
|-- .gitignore      # what Git ignores
|-- README.md       # overview + start commands
|-- poc1_streamlit/
|   |-- .venv/      # local only (ignored)
|   |-- app.py
|   |-- sample_data.py
|   +-- requirements.txt
|-- poc2_fastapi_sqlite/
|   |-- backend/    # FastAPI + SQLite
|   |-- frontend/   # Streamlit
|   +-- requirements.txt
+-- poc3_xgboost_churn/   # like POC 2 + model
```

Conventions

- one repo, one folder per POC
- own `.venv` + `requirements.txt` per POC
- root `README.md` with start commands

What belongs in the repo?

Everything needed to rebuild the project elsewhere – nothing generated, nothing secret.

.gitignore: what stays out of the repository

A text file in the repo root that tells Git which files and folders to **ignore**. Without it, repositories become bloated, slow – and unsafe.

Entry	Why
`.venv/`, `__pycache__/`	local, machine-specific
`*.db`, `data/uploads/`	generated at runtime
`.env`, `*.key`	secrets – once pushed = leaked

Example `.gitignore`

```
.venv/
__pycache__/
*.db
data/uploads/
.env
*.key
.streamlit/secrets.toml
```

Check `.gitignore` *before* the first commit – ignoring a file later does not remove it from history.

Secrets and security

API keys and passwords never go into code or Git

Never in the repository

API keys, tokens, passwords, `.env` files with real values, databases with personal data

Instead

Keep secrets in a local `.env` (ignored). Commit a `.env.example` with empty placeholders.

```
# .env.example
LLM_API_KEY=
```

If it happened anyway

1. **revoke or rotate the key at once** (course key: tell the lecturer) – deleting the file is not enough
2. then clean the history (`git filter-repo`) or ask for help

GitHub helps a little

Secret scanning and push protection catch many known key formats in public repos – but not all.

Virtual environments

Every project gets its own, isolated set of packages

Why?

Projects need different packages and versions. A `.venv` keeps them apart and makes `requirements.txt` reproducible.

In VS Code

`Ctrl+Shift+P` →
Python: Create Environment
or *Python: Select Interpreter*

```
# create (once per project)
python3.13 -m venv .venv        # macOS / Linux
py -V:3.13 -m venv .venv        # Windows

# activate (every new terminal)
source .venv/bin/activate       # macOS / Linux
.venv\Scripts\Activate.ps1      # Windows

# install: by name, or all from the list
pip install streamlit pandas
pip install -r requirements.txt
```

The terminal line starts with `(.venv)`? Then you are in the right environment.

Anatomy of a good coding prompt

Six building blocks that turn generic code into *your* code

1. **Goal** – what should work, for whom
2. **Stack** – FastAPI + SQLAlchemy + SQLite
3. **Files** – `backend/main.py`, `frontend/app.py`
4. **Interfaces & output** – tables, endpoints, JSON fields, UI elements
5. **Demo data** – `sample_data.py`, sample button
6. **Done-criteria** – "done when ...", packages, ports

Why each block matters

Every block you leave out is a decision the agent makes for you – usually the most generic one.

Done-criteria example

"Done when `streamlit run app.py` starts without errors and every numeric column has a histogram."

Bad prompts produce generic code. Good prompts produce *your* code.

From a vague to a specific prompt

Vague prompt – leads to generic code

```
Build me an app that can analyse CSVs and does a bit of ML.
```

Specific prompt – leads to *your* code

Create `backend/` with FastAPI + SQLAlchemy + SQLite. **GOAL** **STACK** **FILES**

Models: `Upload`, `AnalysisRun`, `ColumnStat`. Endpoints: `POST /api/upload`, `GET /api/uploads`, `POST /api/analyze/{id}`. Pydantic schemas for all responses. **INTERFACES**

Demo data via `sample_data.py`. **DEMO DATA**

Done when all endpoints work in `/docs` with the sample upload. **DONE-CRITERIA**

The second prompt is longer – and saves you three rounds of corrections.

Project instructions: tell the agent your rules once

A Markdown file that Copilot reads with every chat request in this repository

.github/copilot-instructions.md

```
# Rules for this repository
- Python 3.13, one .venv per POC folder.
- Keep code simple and commented for
  business students.
- Ask before adding a new dependency;
  keep requirements.txt up to date.
- Never hard-code secrets; use .env.
- After each change: explain it in
  3-5 bullets and update the README
  run commands.
```

Why?

Conventions you would otherwise repeat in every prompt – stated once, applied always.

Alternatives

`AGENTS.md` in the repo root works too. The chat command `/init` drafts a starter file for you.

Create this file in your repository now –
short, concrete house rules the agent reads in every POC.

Plan first, then build

For every step that touches more than one file



What to look for in a plan

right files? right stack? no surprise dependencies? steps you understand? a way to test?

Correct in plain language

"Use SQLite, not PostgreSQL. Put the API in `backend/`. Skip authentication."

Correcting a plan costs one sentence; correcting finished code costs an afternoon.

Three-stage review for AI-generated code

1 Read & understand

Do I understand *every* changed line? If not: ask in Ask – "explain lines 12–18" – *then* accept. Use the diff view.

2 Run & test

Start the app. Try the normal case ("happy path") *and* at least one error case: empty CSV, wrong file type, missing field.

3 Hallucination check

Do the packages exist? Does `pip install` work? Are paths, ports and parameters consistent?

Read it, run it, check it – only then commit it.

Typical hallucinations in AI-generated code

Plausible-looking, but wrong

Symptoms

- invented packages → `ModuleNotFoundError`
- non-existent functions, outdated APIs
- plausible but wrong defaults or column names
- gaps: `# implementation here`
- "it works" – without having run it

A security angle

Don't install a package just because the AI named it – attackers register look-alike names. Check `pypi.org` first.

Countermeasure

Paste the full error back and ask the agent to fix the *cause*.

If the code sounds too good – install and run it first, *then* believe it.

Top pitfalls – and how to avoid them

	Pitfall	Prevention
1	wrong Python environment	`(.venv)` in the terminal; *Python: Select Interpreter*
2	secrets in the repository	`.env` in `.gitignore` before the first commit
3	"connection refused" in POC 2/3	the backend is not running – start it in a second terminal
4	port already in use (8000 / 8501)	stop the old process (`Ctrl+C`) or use another port
5	huge, unexplained changes	plan first; one feature per prompt; ask for a summary
6	no commits for hours	commit after every working step

Setup before code: most problems appear before the first prompt – or when two processes must talk.

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

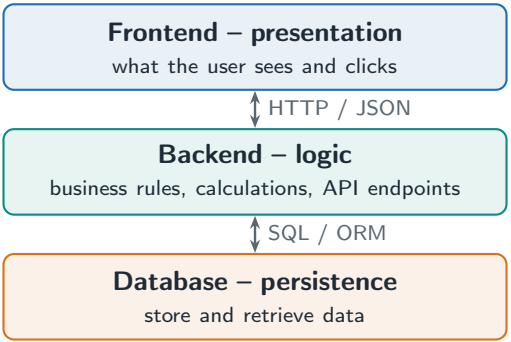
Part 4 • Session 2 and following: How apps are built

Frontend, backend, database – the vocabulary every automation project uses.

- ✔ Explain the three-tier architecture with an everyday analogy
- ✔ Follow a click from the browser to the database and back
- ✔ Read an HTTP request, a JSON response and a simple SQL query
- ✔ Know what runs where: processes, ports, localhost

The three-tier architecture

Almost every business application is built in three layers



Restaurant analogy

Dining room = frontend
Kitchen = backend
Pantry = database

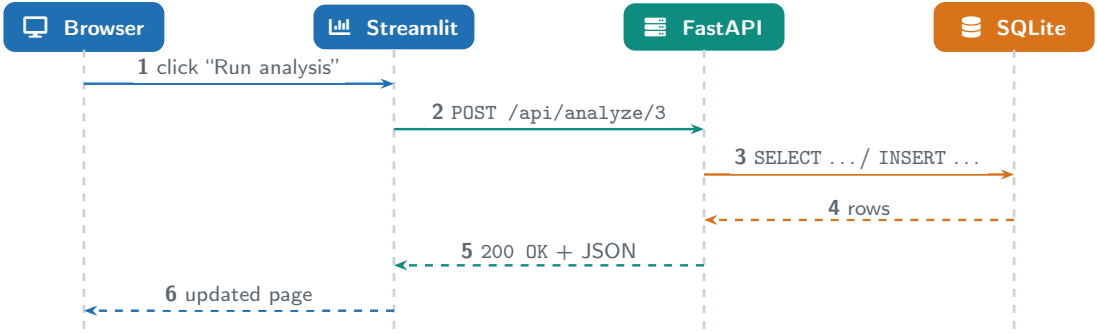
The waiter (API) carries orders in a fixed format.

Typical tools:
Streamlit, React
FastAPI, Django
SQLite, PostgreSQL

Architecture answers one question: who calls whom, how – and who stores what.

What happens when you click a button?

One request through all three layers (POC 2)



Browser ↔ Streamlit: WebSocket • Streamlit ↔ FastAPI: HTTP • FastAPI ↔ SQLite: SQL via SQLAlchemy

Every layer only talks to its neighbour – that is what makes each one replaceable.

HTTP in one slide

The language in which frontends and backends talk

Methods – what you want

GET	retrieve data (no changes)
POST	submit data, create something
PUT	replace an existing resource
DELETE	delete a resource

A request

```
POST http://localhost:8000/api/predict
Content-Type: application/json

{"tenure_months": 3, "monthly_charges": 79.9}
```

Status codes – what happened

200 OK	success
201 Created	new resource created
400 Bad Request	the request is malformed
404 Not Found	wrong path or unknown ID
422 Unprocessable	valid JSON, wrong fields/types
500 Server Error	bug in the backend

2xx = fine
4xx = the caller made a mistake
5xx = the server did

Source: MDN Web Docs (HTTP methods, status codes)

An API is a contract

Both sides agree on paths, fields and types – and can then evolve independently

Request (frontend → backend)

```
POST /api/predict
{
  "age": 34,
  "tenure_months": 7,
  "monthly_charges": 74.9,
  "contract_type": "monthly"
}
```

Response (backend → frontend)

```
200 OK
{
  "churn_probability": 0.71,
  "prediction": 1,
  "model_run_id": 3
}
```

FastAPI turns this contract into a live, clickable documentation page at `/docs`.

Agree on the contract first – then build (and prompt) both sides separately.

Frontend vs. backend – and where Streamlit sits

	Frontend	Backend
Job	show data, collect input	rules, calculations, data access
Talks to	the user	database, models, other systems
Must be	easy to use	correct, secure, reliable
In POC 2/3	Streamlit	FastAPI

Streamlit is special

You write only Python. The script runs **on the server**; the browser just displays the result and sends clicks back (WebSocket).

Consequence

Uploaded data and the app's session values (`st.session_state`) live in the **server's memory** – one session per browser tab.

Streamlit gives you a frontend without HTML or JavaScript – ideal for data prototypes.

Databases in five minutes

Tables, SQL – and an ORM that writes SQL for you

Table `uploads`

id	filename	n_rows	n_cols
1	sample.csv	500	8
2	sales_q3.csv	1 204	12

id = primary key: unique for every row

SQL: ask the database

```
SELECT filename, n_rows FROM uploads
WHERE n_rows > 1000;
```

SQLite

a complete SQL database in a **single file** (`app.db`) – no server, no configuration, part of Python.

ORM (SQLAlchemy)

maps Python classes to tables: `Upload(filename=...)` becomes a row – the ORM writes the SQL.

Tables store facts, SQL asks questions, the ORM lets Python do both.

Processes, ports and localhost

Two programs, two terminals, two addresses

Terminal 1 – backend

```
uvicorn backend.main:app --reload
# -> http://127.0.0.1:8000
```

Terminal 2 – frontend

```
streamlit run frontend/app.py
# -> http://localhost:8501
```

process	a running program – it keeps running until you stop it with `Ctrl+C`
localhost (= `127.0.0.1`)	"this computer". FastAPI listens only there; Streamlit also shows a *Network URL*
port	a numbered door of the computer: Streamlit 8501, FastAPI 8000 by default

Both processes must run at the same time – one terminal each.

Why we separate layers

Benefits and costs

+ Benefits

- **clarity** – each layer has one purpose
- **maintainability** – change the UI without touching the logic
- **reuse** – the same API serves web, mobile, scripts, other systems
- **scalability** – the backend can serve many clients

− Costs

- more moving parts
- communication over HTTP
- two processes to start and watch
- a contract (API) to keep in sync

Start small (POC 1) – add layers only when you need them (POC 2, 3).

The role of each technology

One sentence each

Technology	Role in our POCs	POC
Streamlit	frontend in pure Python – ideal for data apps	1, 2, 3
pandas	tables in Python: read CSVs, statistics	1, 2, 3
FastAPI + Pydantic + Uvicorn	backend: REST API with automatic validation and docs	2, 3
SQLite + SQLAlchemy	database in one file, accessed via Python classes	2, 3
XGBoost + scikit-learn	train, evaluate and serve a classification model	3
Git / GitHub	versions, backup, collaboration	all
React, Vue, ...	professional JavaScript frontends – outlook only	–

Everything is Python – one language from the data to the user interface.

How the three POCs build on each other

POC 1 – standalone

- Streamlit app
- CSV upload / sample data
- descriptive statistics
- charts

one process, one file

POC 2 – three tiers

- Streamlit frontend
- FastAPI backend
- SQLite database
- REST API in between

same use case, professional structure

POC 3 – plus ML

- Streamlit + FastAPI
- SQLite database
- XGBoost classifier
- churn prediction + history

data + model + UI = ML app

Each step adds architecture – nothing is taken away.

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

Part 5 • Session 2 and following: POC 1 – A data app with Streamlit

One file, one process – a useful data tool in under an hour.

- ✔ Understand Streamlit's rerun model and session state
- ✔ Build the app with one precise agent prompt
- ✔ Verify it against acceptance criteria and explain the code

Use case: understand a CSV in 30 seconds

Scenario

You receive a CSV export – sales, HR, logistics. First you want to know:

- how many rows and columns?
- which data types, how many missing values?
- how are the values distributed?

Goal

One Streamlit app: start it, upload a CSV (or load sample data), get a basic profile and charts immediately.

Deliberately without

backend, database – both come in POC 2.

Data profiling starts every analytics or automation project – worth automating.

Streamlit in 60 seconds

A web app is just a Python script

```python
import pandas as pd
import streamlit as st

st.title("CSV Explorer")
file = st.file_uploader("Upload a CSV", type="csv")

if file is not None:
    df = pd.read_csv(file)
    st.metric("Rows", len(df))
    st.dataframe(df.describe())
    col = st.selectbox("Column", df.columns)
    st.bar_chart(df[col].value_counts())
```

Run it: `streamlit run app.py` → opens `http://localhost:8501`

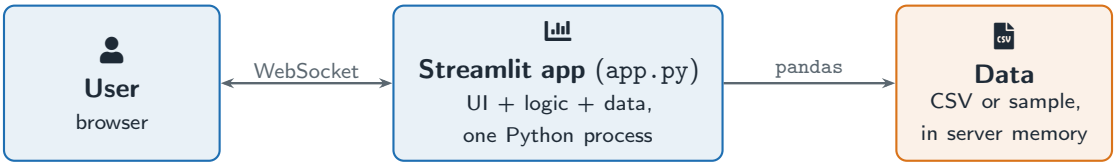
Three ideas to remember

1 **Rerun:** every click reruns the script top to bottom.

2 **Session state:** `st.session_state` keeps values between reruns.

3 **Caching:** `@st.cache_data` avoids reloading data on every rerun.

POC 1 architecture: everything in one process



One process, one language, one file

no separate backend • no database
ideal to learn Streamlit
and the coding loop

Consequence

Data lives only in the Streamlit server's memory, per session: close the tab or restart – and it is gone.

POC 1: steps in agent mode

1. Open your **repository root** in VS Code; create folder `poc1_streamlit/`
2. Terminal: `cd poc1_streamlit`, create + activate `.venv`, select it as interpreter
3. Chat: *Agent*, *Manual permissions*; paste the prompt (next slide)
4. Approve the agent's `pip install` and `streamlit run app.py`
5. Run the acceptance tests; iterate; **commit & push**

Timing

prompt + generation: 5 min
review + tests: 15 min
understanding + extensions: rest of the session

Tip

Keep the chat open and the app visible side by side – you will switch often.

POC 1: the prompt

Prompt for Copilot – Agent

Work only inside `poc1_streamlit/` and use its existing `.venv`. Create a Streamlit app `app.py` for a quick first look at a CSV file. Data sources: (a) `st.file_uploader` for my own CSV, (b) a button "Load sample data".

Put the sample data in `sample_data.py`: a function `get_sample_df()` that generates about 500 realistic rows with numpy/pandas (mixed numeric and categorical columns, some missing values).

After loading, show: (1) number of rows and columns, (2) data type and missing values per column, (3) `df.describe()`, (4) a histogram and a box plot (matplotlib) for every numeric column.

Create `requirements.txt` (streamlit, pandas, numpy, matplotlib), `.gitignore` and a short README; install the requirements and start the app.

Done when `streamlit run app.py` works with the sample data. Finally, explain what you created in five bullet points.

GOAL STACK FILES DEMO DATA OUTPUT DONE-CRITERIA

POC 1: run and verify

Acceptance tests – tick them before you commit

Happy path

- ☑ app starts without errors
- ☑ "Load sample data" shows about 500 rows
- ☑ missing values per column are plausible
- ☑ every numeric column has a histogram and a box plot

Error cases & checks

- ☑ upload an empty CSV
- ☑ upload a file with only text columns
- ☑ upload a semicolon-separated CSV (Excel export)
- ☑ `requirements.txt` matches the imports

Found a problem?

Describe it precisely: "When I upload an empty CSV, the app crashes with *EmptyDataError*. Catch it and show a friendly message."

A POC is done when it passes your tests – not when the agent says it is done.

POC 1: understand your code

Five questions to ask Copilot in *Ask*

1. "Walk me through `app.py` from top to bottom. What happens when I click the sample button?"
2. "Why does the app need `st.session_state` here – or why not?"
3. "How does `get_sample_df()` create missing values?"
4. "What does `df.describe()` calculate, and for which columns?"
5. "Which line would I change to use 30 histogram bins?"

Self-test

Close the chat and explain the app to your neighbour in two minutes. Can you?

Ask (Local target) cannot change files – explore without risk.

If you can't explain it, you don't own it yet.

POC 1: extensions

For those who finish early – one feature per prompt

EASY **Quick wins**

- sidebar filter for a categorical column
- correlation heatmap of numeric columns
- cache loading with `@st.cache_data`

HARDER **Mini automation**

- download a data-quality report (Markdown/CSV)
- automatic warnings: >20 % missing, constant columns, duplicates
- compare two uploaded CSVs

Each extension = one prompt, one review, one test, one commit.

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

Part 6 • Session 2 and following: POC 2 – Frontend, backend and database

Same use case, professional architecture: three layers, two processes, one API.

- ✔ Build a REST API with FastAPI and store results in SQLite
- ✔ Connect a Streamlit frontend to the API over HTTP
- ✔ Run, test and debug two processes that talk to each other

Same use case, new architecture

POC 1 and POC 2 solve the same problem – only the structure changes

What stays the same

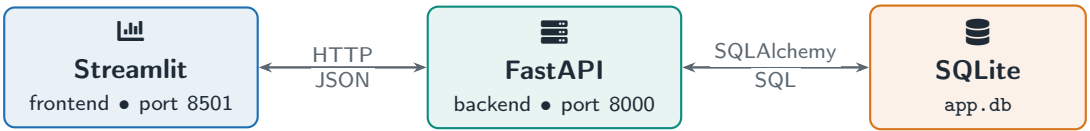
- use case: CSV → statistics → charts
- Streamlit as user interface
- pandas for the analysis

What changes

- 1 process → **2 processes**
- memory only → **SQLite database**
- function call → **HTTP API**
- one file → `frontend/` + `backend/`
- no history → **all runs retrievable**

Architecture is "who calls whom, how" – the business logic stays the same.

POC 2 architecture



Streamlit	UI: data source, upload, buttons, tables, charts – knows only the API URLs
FastAPI	logic: parse CSVs, compute statistics, validate input, read/write the database
SQLite	persistence: upload metadata, analysis runs, column statistics

The frontend never touches the database – everything goes through the API.

FastAPI in 60 seconds

An endpoint is a Python function with a URL

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Churn API")
class Customer(BaseModel):          # the contract
    tenure_months: int
    monthly_charges: float

@app.get("/api/health")
def health():
    return {"status": "ok"}

@app.post("/api/predict")
def predict(c: Customer):
    risk = 0.8 if c.tenure_months < 6 else 0.2
    return {"churn_probability": risk}
```

What FastAPI does for you

- turns functions into HTTP endpoints
- validates JSON against the Pydantic model (wrong type → 422)
- generates interactive docs at /docs

```
uvicorn main:app --reload
```

SQLite and SQLAlchemy in 60 seconds

A Python class becomes a database table

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, Mapped
from sqlalchemy.orm import mapped_column, Session

class Base(DeclarativeBase):
    pass
class Upload(Base):                # table "uploads"
    __tablename__ = "uploads"
    id: Mapped[int] = mapped_column(primary_key=True)
    filename: Mapped[str]
    n_rows: Mapped[int]

engine = create_engine("sqlite:///./app.db")
Base.metadata.create_all(engine)    # create tables
with Session(engine) as session:
    session.add(Upload(filename="sample.csv", n_rows=500))
    session.commit()
```

Read it like this

class = table
attribute = column
object = row
session = a unit of work: add, commit, query

Want to look inside `app.db`? Install a SQLite viewer extension in VS Code.

POC 2: project structure

```
poc2_fastapi_sqlite/
|-- .venv/                # one venv for both parts
|-- backend/
|   |-- __init__.py       # makes backend a package
|   |-- main.py           # FastAPI app + endpoints
|   |-- models.py         # SQLAlchemy tables
|   |-- schemas.py        # Pydantic models (contract)
|   |-- database.py       # engine + session
|   +-- sample_data.py    # demo dataset
|-- frontend/
|   +-- app.py            # Streamlit, calls the API
|-- data/uploads/         # stored CSV files (ignored)
|-- app.db                # created at start (ignored)
+-- requirements.txt
```

Separation of concerns

Each backend file has one job – easy to read, easy to prompt ("change only `models.py`").

Run from the project folder

```
uvicorn backend.main:app
streamlit run frontend/app.py
```

POC 2: the API contract

Endpoints the frontend may call

Method	Path	Purpose
POST	`/api/upload`	receive a CSV, store file + metadata, return `upload_id`
POST	`/api/sample`	create (or return) the sample upload – testing without a file
GET	`/api/uploads`	list all uploads with timestamp
POST	`/api/analyze/{id}`	start an analysis run, compute and store statistics
GET	`/api/stats/{id}`	descriptive statistics per column
GET	`/api/plots/{id}`	histogram data per numeric column



Test every endpoint in `http://localhost:8000/docs` before you build the frontend.

POC 2: database tables

Table	Columns (example)
`uploads`	`id`, `filename`, `n_rows`, `n_cols`, `uploaded_at`
`analysis_runs`	`id`, `upload_id` (→ `uploads`), `started_at`, `finished_at`, `status`
`column_stats`	`id`, `run_id` (→ `analysis_runs`), `column`, `dtype`, `n_missing`, `mean`, `std`, `min`, `max`

The payoff

Close the app, come back tomorrow – all uploads and results are still there.

Relations

One upload → many runs; one run → one stats row per column.

POC 2: steps in agent mode

Plan first – this step touches many files

1. Repo root open; create `poc2_fastapi_sqlite/`; own `.venv`, activated and selected
2. *Plan* (`/plan`): backend prompt, review the plan, *Implement Plan*
3. Start the backend, test all endpoints in `/docs`, **commit**
4. Frontend prompt in *Agent*; start Streamlit in a second terminal
5. Test the full chain, iterate, **commit & push**

Backend first

A backend you have tested in `/docs` makes frontend bugs easy to locate.

Check the plan for

SQLite (not PostgreSQL), no login, no CORS, the files from the structure slide

POC 2: prompt 1 – backend

Prompt for Copilot – /plan, then Implement Plan

Work only inside `poc2_fastapi_sqlite/` and use its existing `.venv`. Create a FastAPI backend in `backend/` (a package with `__init__.py`) using SQLAlchemy 2.0 style and SQLite (`app.db`). Files: `database.py`, `models.py` (Upload, AnalysisRun, ColumnStat), `schemas.py` (Pydantic), `sample_data.py` (about 500 realistic demo rows), `main.py`.

Endpoints: POST `/api/upload` (store the CSV in `data/uploads/`, save metadata, return `upload_id`), POST `/api/sample` (create or return the sample upload, return its `upload_id`), GET `/api/uploads`, POST `/api/analyze/{id}`, GET `/api/stats/{id}`, GET `/api/plots/{id}` (histogram data per numeric column).

Use a lifespan function to create the tables and seed the sample upload. No CORS middleware. `requirements.txt`: `fastapi[standard]`, sqlalchemy, pandas – install it. Start command: `uvicorn backend.main:app --reload`.

Done when every endpoint works in `/docs`.

Why the backend prompt works

Stack and structure fixed

FastAPI, SQLAlchemy 2.0, SQLite, file names – no guesswork for the agent.

Endpoints defined

Paths and methods are the contract the frontend prompt will rely on.

Data model given

Three tables structure uploads, runs and column statistics.

Testable from minute one

Seeded sample upload + `/docs` = a done-criterion you can check.

A good backend prompt reads like a mini specification – because it is one.

POC 2: prompt 2 – frontend

Prompt for Copilot – Agent

Create `frontend/app.py` (Streamlit) in `poc2_fastapi_sqlite/` that talks to the backend **only** via HTTP with `requests`; base URL from the environment variable `BACKEND_URL` (default `http://localhost:8000`).

A `st.radio` selects the data source: "My CSV" (`st.file_uploader` + button "Upload" → POST `/api/upload`, each file only once) or "Sample data" (button "Load sample data" → POST `/api/sample`). Store the returned `upload_id` in `st.session_state`.

A button "Run analysis" calls POST `/api/analyze/{id}`; show GET `/api/stats/{id}` as a table and GET `/api/plots/{id}` as histograms. A sidebar lists earlier uploads (GET `/api/uploads`).

If the backend is not reachable, show a clear error message. Add streamlit, requests and matplotlib to `requirements.txt` and install them. Done when "Load sample data" → "Run analysis" shows the table and the histograms.

Why the frontend prompt works

The frontend knows URLs – nothing else

```python
import os
import requests
import streamlit as st

BACKEND_URL = os.getenv("BACKEND_URL", "http://localhost:8000")      # configurable

if st.button("Load sample data"):
    r = requests.post(f"{BACKEND_URL}/api/sample", timeout=10)       # HTTP call
    r.raise_for_status()                                             # 4xx/5xx -> error
    st.session_state["upload_id"] = r.json()["upload_id"]            # survives reruns
```

Separation & state

No import from `backend/` – the frontend could be replaced by a mobile app or a script. `st.session_state` keeps the `upload_id` for the next click.

POC 2: run and verify

Two terminals – then the acceptance tests

Terminal 1 – backend

```
cd poc2_fastapi_sqlite
source .venv/bin/activate
uvicorn backend.main:app --reload
```

then open `http://localhost:8000/docs`

Windows: `.venv\Scripts\Activate.ps1`

Terminal 2 – frontend

```
cd poc2_fastapi_sqlite
source .venv/bin/activate
streamlit run frontend/app.py
```

opens `http://localhost:8501`

Windows: `.venv\Scripts\Activate.ps1`

Acceptance tests

- ☑ all six endpoints work in `/docs`
- ☑ sample data → analysis → table + charts
- ☑ restart both – old uploads still listed
- ☑ stop the backend – friendly error in the UI

Debugging the three layers

Where to look when something breaks

Symptom	Likely layer	What to do
"connection refused"	backend not running	start Uvicorn in terminal 1; check the port
404 Not Found	wrong path	compare the URL with `/docs`
405 Method Not Allowed	wrong method	GET used on a POST endpoint (or vice versa)
422 Unprocessable Content	contract mismatch	read the error body: which field is wrong?
500 Internal Server Error	backend bug	read the traceback in terminal 1, paste it to the agent
empty list after restart	database path	started from another folder? `./app.db` is relative
"address already in use"	old process	stop it with `Ctrl+C` or use another port

Debug from the bottom up: database → API in `/docs` → frontend.

POC 2: extensions

EASY **Quick wins**

- delete an upload (DELETE `/api/uploads/{id}`)
- show the run history per upload
- download the statistics as CSV

HARDER **Towards production**

- automatic data-quality rules stored per upload
- a small Python script that calls the API – no UI
- tests for two endpoints with FastAPI's `TestClient`

The script-without-UI extension shows why APIs matter for automation: machines can use them too.

Outline

1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

Part 7 • Session 2 and following: POC 3 – Adding a machine-learning model

From descriptive statistics to predictions: data → model → API → UI.

- ✔ Train and evaluate an XGBoost classifier on churn data
- ✔ Serve predictions through the FastAPI backend and log them
- ✔ Judge a model with the right metrics – and use it responsibly

What changes from POC 2 to POC 3

What stays the same

- three-tier architecture
- REST API with Pydantic schemas
- persistence in SQLite
- Streamlit frontend calling the API

What is added

- a **model layer**: `model.json` + metadata
- data generator and training script
- prediction endpoint returning a probability
- prediction history (audit trail)
- model-run table with metrics

An ML app is a three-tier app plus careful model management.

Use case: predict customer churn

Scenario

A telecom provider wants to know which customers are likely to cancel their contract (*churn*) and contact them proactively. The sales team needs a web app to:

- **browse** existing customer data
- get a **churn probability** per customer
- keep a **log** of all predictions

Why this is a good ML example

- clear **binary** target: churn yes/no
- realistic mix of numeric and categorical features
- direct business value, easy to motivate
- small enough to build end to end in one session

Retention is usually cheaper than acquisition – if you know whom to call.

The data

One row per customer – synthetic, but realistic

id	age	contract_type	tenure_months	monthly_charges	payment_method	support_calls	churn
1001	34	monthly	7	74.90	credit card	3	1
1002	58	two years	48	99.50	bank transfer	0	0
1003	27	monthly	2	59.95	credit card	4	1

```
features = ["age", "tenure_months", "monthly_charges",
            "contract_type", "payment_method",
            "support_calls"]
target   = "churn"      # 0 = stays, 1 = cancels
```

Why synthetic data?

No personal data, no licence questions – and we control how features relate to churn (about 20 % churn rate). The agent generates it with numpy.

Machine learning in 60 seconds

Learn from labelled examples, test on unseen ones

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score
from xgboost import XGBClassifier

df = pd.read_csv("data/customers.csv")
X = pd.get_dummies(df.drop(columns=["id", "churn"]))
y = df["churn"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

model = XGBClassifier(n_estimators=300, max_depth=4,
                      learning_rate=0.05)
model.fit(X_train, y_train)                    # learn
proba = model.predict_proba(X_test)[:, 1]      # predict
print("ROC-AUC:", roc_auc_score(y_test, proba))
model.save_model("backend/model.json")         # persist
```

The recipe

1. encode categories (one-hot)
2. split: 80 % train, 20 % test
3. fit the model
4. evaluate on the test set
5. save the model

The output is a **probability**, not a verdict.

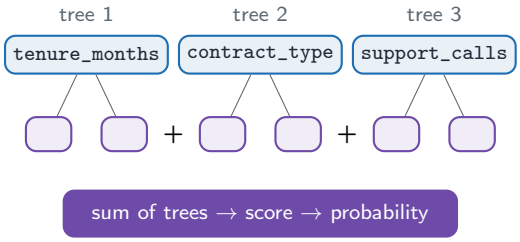
Why XGBoost?

A strong default for tabular business data

Gradient-boosted decision trees

Many small decision trees, trained one after another – each new tree corrects the errors of the previous ones.

- strong default on **tabular data** – often better than neural networks there
- handles **missing values** natively
- **feature importance** for a first explanation
- fast to train on a laptop, a few lines of code



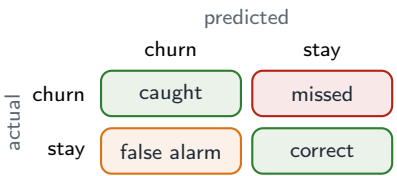
Source: Chen & Guestrin (2016); Grinsztajn et al. (2022)

Is the model any good?

Accuracy alone can fool you

The accuracy trap

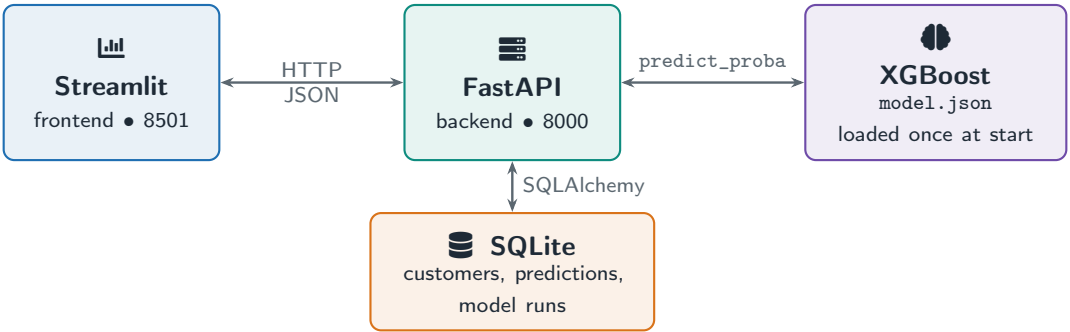
With 20 % churn, a "model" that always says *no churn* is right 80 % of the time – and useless.



Metric	Question it answers
Precision	Of the customers we call, how many would really leave?
Recall	Of all real churners, how many do we catch?
ROC-AUC	How well does the score rank churners above stayers? (0.5 = random)
Baseline	What does "always no churn" achieve?

A missed churner costs a customer, a false alarm a discount – the threshold is a business decision.

POC 3 architecture



Data flow of one prediction

Streamlit → `POST /api/predict` → FastAPI builds the feature vector → model returns a probability → row in `predictions` → JSON back → Streamlit shows the score.

POC 3: project structure

```
poc3_xgboost_churn/
|-- .venv/
|-- backend/
|   |-- __init__.py
|   |-- main.py              # FastAPI app + endpoints
|   |-- models.py            # SQLAlchemy tables
|   |-- schemas.py           # Pydantic models
|   |-- database.py          # engine + session
|   |-- generate_data.py     # synthetic churn data
|   |-- train_model.py       # training + evaluation
|   |-- model.json           # trained XGBoost model
|   +-- model_meta.json      # feature columns, metrics
|-- frontend/app.py          # Streamlit UI
|-- data/customers.csv       # generated
|-- app.db                   # generated (ignored)
+-- requirements.txt
```

New files

`generate_data.py`, `train_model.py`, `model.json`, `model_meta.json`

Training–serving skew

The backend must build *exactly* the columns the model was trained on – hence `model_meta.json`.

POC 3: API endpoints and tables

Method	Path	Purpose
GET	`/api/customers`, `/api/customers/{id}`	list / details of stored customers
POST	`/api/predict`	prediction for a feature vector (JSON)
POST	`/api/predict/{customer_id}`	prediction for a stored customer
GET	`/api/sample-customer`	a random example profile for testing
POST	`/api/train`	retrain the model (optional)
GET	`/api/model/info`, `/api/predictions`	model metrics; prediction history

Table	Columns (example)
`customers`	`id, age, contract_type, tenure_months, monthly_charges, payment_method, support_calls, churn`
`predictions`	`id, customer_id, probability, prediction, model_run_id, created_at`
`model_runs`	`id, trained_at, n_samples, accuracy, roc_auc, precision, recall, model_path`

POC 3: steps in agent mode

1. Repo root open; create `poc3_xgboost_churn/`; own `.venv`, activated and selected
2. **Data & model:** prompt 1 – run training, read the metrics, **commit**
3. **Backend:** prompt 2 (`/plan` → Implement Plan) – test in `/docs`, **commit**
4. **Frontend:** prompt 3 – start both processes, test, **commit**
5. Iterate: feature importance, threshold, batch scoring

Checkpoint after step 2

Does the model beat the "always no churn" baseline? If not, fix the data before building an API.

Reuse

Tell the agent: "Use the same structure and conventions as `poc2_fastapi_sqlite`."

POC 3: prompt 1 – data and model

Prompt for Copilot – Agent

Work only inside `poc3_xgboost_churn/` and use its existing `.venv`. Create `backend/generate_data.py`: generate a realistic synthetic churn dataset with numpy (about 5,000 rows; fields `id`, `age`, `tenure_months`, `monthly_charges`, `contract_type`, `payment_method`, `support_calls`, `churn`; churn rate about 20 %, plausibly related to the features) and save it as `data/customers.csv`. Create `backend/train_model.py`: run the generator if the CSV is missing; one-hot encode the categorical columns with pandas; stratified 80/20 split; train an `XGBClassifier`; print accuracy, ROC-AUC, precision, recall and the baseline of always predicting "no churn". Save the model with `save_model("backend/model.json")` and the feature columns plus metrics to `backend/model_meta.json`. Use package imports (`backend.generate_data`). Create `requirements.txt` (numpy, pandas, scikit-learn, xgboost), install it and run `python -m backend.train_model`. Done when the metrics are printed and `model.json` exists.

POC 3: prompt 2 – backend

Prompt for Copilot – /plan, then Implement Plan

Create the FastAPI backend in `poc3_xgboost_churn/backend/` with the same structure and conventions as `poc2_fastapi_sqlite`. SQLAlchemy 2.0 models: Customer, Prediction, ModelRun. Endpoints: GET `/api/customers`, GET `/api/customers/{id}`, POST `/api/predict`, POST `/api/predict/{customer_id}`, GET `/api/sample-customer`, POST `/api/train`, GET `/api/model/info`, GET `/api/predictions`. In a lifespan function: create the tables, import `data/customers.csv` if the customers table is empty, load `model.json` and `model_meta.json` once, and create a ModelRun from the metadata if none exists. For each prediction, build the one-hot columns exactly as in `model_meta.json`, store the prediction with the model run and return `churn_probability`, `prediction` and `model_run_id`. `POST /api/train` adds a new ModelRun and reloads the model. Use `Literal[...]` for the categories. No CORS middleware. Add `fastapi[standard]` and `sqlalchemy` to `requirements.txt` and install them. Done when every endpoint works in `/docs`.

POC 3: prompt 3 – frontend

Prompt for Copilot – Agent

In `poc3_xgboost_churn/`, create `frontend/app.py` (Streamlit, calls the backend only via HTTP, `BACKEND_URL` as in POC 2) with three tabs:

Customers: table from GET `/api/customers` with a filter by `contract_type`.

Prediction: a form with all features and sensible defaults, a button "Load sample customer" (GET `/api/sample-customer` fills the form) and a selectbox "Choose existing customer" (POST `/api/predict/{customer_id}`). The button "Predict" calls POST `/api/predict` and shows the probability as a metric and a progress bar.

History: the latest predictions from GET `/api/predictions`, plus the model metrics from GET `/api/model/info`.

Add streamlit and requests to `requirements.txt` and install them. Done when all three tabs work with the sample customer.

POC 3: run and verify

Sanity checks for an ML app

Model

- ☑ ROC-AUC clearly above 0.5; recall reported
- ☑ model beats the "always no churn" baseline
- ☑ `/api/model/info` shows the metrics

Behaviour

- ☑ high-risk profile → high probability
- ☑ loyal two-year customer → low probability

System

- ☑ every prediction appears in the history
- ☑ invalid input in `/docs` (text as age) → 422, no crash
- ☑ restart: model and data still there

Too good to be true?

ROC-AUC near 1.0 on synthetic data usually means the generator made churn too predictable.

From prediction to action

A score is not a decision

Who acts?

The model ranks; people decide whom to contact and with which offer. Define the process around the score.

Fair & explainable?

Would the model treat groups unfairly? Can we explain a score to a manager or a customer?

Data protection

Real customer data is personal data: purpose limitation, access control, deletion – GDPR applies.

Automation creates value only when the prediction is embedded in a responsible business process.

POC 3: extensions

EASY **Quick wins**

- feature-importance chart in the History tab
- threshold slider: who gets called?
- colour the probability (green / amber / red)

HARDER **Closer to reality**

- batch scoring: upload a CSV, download scores
- compare two model runs before switching
- train on the IBM Telco churn dataset

Batch scoring is how churn models are used in practice: a nightly list for the sales team.

Outline

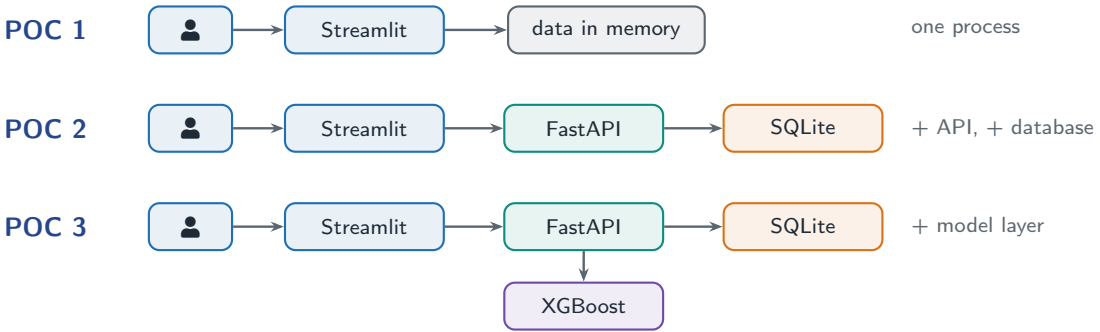
1 From idea to app in ten minutes
2 Setting up your toolchain
3 Working with a coding agent
4 How apps are built
5 POC 1 – A data app with Streamlit
6 POC 2 – Frontend, backend and database
7 POC 3 – Adding a machine-learning model
8 Wrap-up

Part 8 • Session 2 and following: Wrap-up

What you built, what is still missing – and what comes next.

- ✔ Compare the three architectures at a glance
- ✔ Know what separates a POC from a product
- ✔ Take seven habits into every future project

Three POCs at a glance



Each stage *adds* to the architecture – nothing is taken away.

From POC to production

What we deliberately left out – but a product needs

Topic	What a product needs	Who usually owns it
Tests	automated tests for logic and API	development team
Access	login, roles, permissions	IT / security
Deployment	servers or cloud, backups, updates	IT operations
Monitoring	errors, performance, model drift	operations / data science
Data protection	GDPR, data minimisation, deletion	data-protection officer
Ownership	who maintains and pays for it?	business owner

A POC answers *whether* it works. Production answers *how* it keeps working – safely.

Seven take-aways

1. **POC = learning speed over robustness.** Three small apps beat one perfect project.
2. **AI-assisted coding is a loop.** Describe, generate, review, test, commit.
3. **Prompt quality = code quality.** Goal, stack, files, interfaces, demo data, done-criteria.
4. **Read it, run it, check it.** Only then commit it.
5. **Architecture is "who calls whom, how"** – the use case can stay the same.
6. **ML app = three tiers + a model layer** – and the right metrics.
7. **Secrets never go into Git.** `.gitignore` before the first commit.



You now own a professional toolchain – and three prototypes that prove you can use it.

Resources

Course repository

github.com/ChrisW09/Python-for-AI-Driven-Automation

From your first line of Python to AI-driven automation: notebooks, exercises, data and slides.

Documentation

VS Code AI	code.visualstudio.com/docs
GitHub Copilot	docs.github.com/copilot
Streamlit	docs.streamlit.io
FastAPI	fastapi.tiangolo.com
SQLAlchemy	docs.sqlalchemy.org
XGBoost	xgboost.readthedocs.io
Git book	git-scm.com/book

Further reading

Simon Willison's blog on AI-assisted programming: simonwillison.net

MDN Web Docs on HTTP: developer.mozilla.org

Outlook: React, Vue, Svelte – frontend frameworks for products, not used here.

References

Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv:2507.09089.

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proc. KDD '16*, 785–794.

Dell'Acqua, F., et al. (2026). Navigating the jagged technological frontier: Field experimental evidence of the effects of artificial intelligence on knowledge worker productivity and quality. *Organization Science* 37(2), 403–423.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures*. PhD dissertation, University of California, Irvine.

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *NeurIPS 35* (Datasets & Benchmarks).

Karpathy, A. (2025, 2 Feb). "There's a new kind of coding I call 'vibe coding' . . ." [Post on X]. https://x.com/karpathy/status/1886192184808149383

Noy, S., & Zhang, W. (2023). Experimental evidence on the productivity effects of generative artificial intelligence. *Science* 381(6654), 187–192.

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. arXiv:2302.06590.

Willison, S. (2025, 19 Mar). Not all AI-assisted programming is vibe coding (but vibe coding rocks). https://simonwillison.net/2025/Mar/19/vibe-coding/

Questions?

Next: LLMs, RAG & Agentic AI — the building blocks of AI-driven automation

christoph.weisser@hsbi.de

github.com/ChrisW09/Python-for-AI-Driven-Automation

Prof. Dr. Christoph Weisser • AI-Driven Automation • Bielefeld School of Business, HSBI